

Aula – 09 de março de 2026

Comandos iniciais para “limpar” instâncias anteriores no docker.

Caso algum container esteja executando, é necessário pará-lo antes. O comando a seguir para todos os containers em execução:

```
docker stop $(docker ps -aq)
```

Remover todos os containers:

```
docker rm $(docker ps -aq)
```

Remover todos os recursos não utilizados:

```
docker system prune -a --volumes
```

Remove todos os volumes. Apenas use em ambiente de desenvolvimento.

```
docker volume rm $(docker volume ls -q)
```

Extraia o arquivo docker.zip (disponível no Google Classroom) localmente na sua máquina, e depois via terminal, acesse o diretório para onde os mesmos foram extraídos. Estando no diretório correto, execute o comando:

```
docker compose up -d
```

Para acessar:

```
http://0.0.0.0:5601/
```

Ao acessar a tela inicial do Kibana, podemos realizar o login com as seguintes credenciais:

```
usuário: elastic
```

```
senha: user123
```

Em seguida, faremos acesso ao menu Dev Tools, dentro do Kibana:

[Home](#)[Synthetics](#)[User Experience](#)**Security**[Dashboards](#)[Rules](#)[Alerts](#)[Attack discovery](#)[Findings](#)[Cases](#)[Timelines](#)[Intelligence](#)[Explore](#)[Manage](#)**Management**[Dev Tools](#)[Integrations](#)[Fleet](#)[Osquery](#)[Stack Monitoring](#)[Stack Management](#)[Add integrations](#)

Alguns comandos feitos na aula:

Lista os nós do cluster

GET /_cat/nodes

O mesmo que o de cima, mas acrescentando cabeçalho (v = verbose)

GET /_cat/nodes?v

Exibe a “saúde” do cluster (atenção para o status = green, yellow, red)

GET /_cat/health?v

Cria índice chamado wiki_documents1

PUT /wiki_documents1

Obtém detalhes do índice

GET /wiki_documents1

Lista os shards

GET /_cat/shards?v

Lista os índices

GET /_cat/indices?v

Cria índice especificando número de shards e réplicas

PUT /wiki_documents2

```
{
  "settings": {
    "number_of_shards": 2,
    "number_of_replicas": 2,
  }
}
```

Aula – 10 de março de 2026

Começaremos a aula de hoje apagando os índices criados na aula anterior, de modo a termos um ambiente “limpo” novamente.

DELETE wiki_documents1

DELETE wiki_documents2

...

Criando um índice vazio, e adicionando o primeiro documento:

```
PUT /produtos
```

```
POST /produtos/_doc
```

```
{  
  "nome": "cafeteira",  
  "preco": "50",  
  "estoque": 20  
}
```

Adicionando mais documentos através da api bulk:

```
POST /_bulk
```

```
{"index":{"_index": "produtos"}}  
{"nome": "máquina de café espresso", "preco": "300", "estoque": 10}  
{"create":{"_index": "produtos"}}  
{"nome": "moedor de café", "preco": 40, "estoque": 10}
```

Fazendo uma primeira busca “geral”, sem especificar a query:

```
GET /produtos/_search
```

Realizando uma segunda busca, especificando a palavra chave:

```
GET /produtos/_search
```

```
{  
  "query": {  
    "match": {  
      "nome": "café"  
    }  
  }  
}
```

Conforme discutido em aula, o Elasticsearch utiliza o conceito de analyzers para processar campos de texto durante a indexação. Caso nenhuma configuração personalizada seja definida, o standard analyzer é aplicado automaticamente. É possível verificar a saída do standard analyzer nos exemplos a seguir:

```
POST /_analyze
```

```
{  
  "text": "2 guys walk into a bar, but the third... DUCKS! :-)",  
  "analyzer": "standard"  
}
```

POST /_analyze

```
{
  "text": "2 guys walk into a bar, but the third... DUCKS! :-)",
  "char_filter": [],
  "tokenizer": "standard",
  "filter": ["lowercase"]
}
```

Aula – 16 de março de 2026

Começaremos criando um índice com um mapping explícito:

PUT /produtos

```
{
  "mappings": {
    "properties": {
      "nome": {"type": "text"},
      "avaliacao": {
        "properties": {
          "nota": {"type": "integer"},
          "autor": {"type": "text"},
          "descricao": {"type": "text"}
        }
      }
    }
  }
}
```

E faremos a seguinte inserção:

POST /produtos/_doc

```
{
  "nome": "cafeteira",
  "avaliacao": [
    {
      "nota": 5,
      "autor": "Flavio",
      "descricao": "otimo produto"
    },
    {
      "nota": 3,
      "autor": "Jose",
      "descricao": "veio com defeito"
    }
  ]
}
```

```
}  
]  
}
```

Da maneira como criamos, a seguinte query retornará o documento como resultado, mesmo não tendo uma avaliação de Jose com nota maior ou igual a 4.

```
GET /produtos/_search  
{  
  "query": {  
    "bool": {  
      "must": [  
        {  
          "match": {  
            "avaliacao.autor": "Jose"  
          }  
        },  
        {  
          "range": {  
            "avaliacao.nota": {  
              "gte": 4  
            }  
          }  
        }  
      ]  
    }  
  }  
}
```

Este fato ocorre porque o Elasticsearch não armazena a relação entre os elementos internos do vetor. Então, ele não guarda que: nota 3 → autor: Jose → descricao: veio com defeito.

Para resolver este problema, e guardar a relação entre os elementos de um vetor, temos o tipo nested. A criação do índice deve ser alterada da seguinte forma:

DELETE produtos

```
PUT /produtos  
{  
  "mappings": {  
    "properties": {  
      "nome": {  
        "type": "text"  
      },  
      "descricao": {  
        "type": "text"  
      },  
      "avaliacao": {  
        "type": "nested"  
      }  
    }  
  }  
}
```


Tipo keyword:

POST /_analyze

```
{
  "text": "2 guys walk into      a bar, but the third...   DUCKS! :-)",
  "analyzer": "keyword"
}
```

No tipo keyword, nenhum tratamento é feito ao texto, de modo que, o conteúdo original do documento criado será utilizado como entrada única no índice invertido. Para ficar mais claro, observe a saída do analyzer keyword (acima), e compare com a saída do analyzer standard (já visto em sala, e replicado a seguir).

POST /_analyze

```
{
  "text": "2 guys walk into      a bar, but the third...   DUCKS! :-)",
  "analyzer": "standard"
}
```

Tipo array:

O Elasticsearch não implementa o tipo array. Na prática, vetores resultam em dados concatenados “nos bastidores” durante o processo de indexação. Como exemplo, analise os dois comandos a seguir, e perceba que retornam a mesma saída.

String única

POST /_analyze

```
{
  "text": "Strings são concatenadas juntas.",
  "analyzer": "standard"
}
```

Vetor

POST /_analyze

```
{
  "text": ["Strings são", "concatenadas juntas."],
}
```

```
"analyzer": "standard"
}
```

Vamos agora testar alguns exemplos de conversão automática de tipo de dados (coercion). Começaremos criando um novo índice para esta finalidade:

```
PUT /coercion_test
{
  "settings": {
    "index": {
      "number_of_shards": "1",
      "number_of_replicas": "0"
    }
  },
  "mappings": {
    "properties": {
      "preco": {
        "type": "float"
      }
    }
  }
}
```

Inserindo o primeiro documento, e verificando a inserção:

```
POST /coercion_test/_doc
{
  "preco": 7.4
}

GET /coercion_test/_search
```

Testando o funcionamento do mecanismo de coercion:

```
POST /coercion_test/_doc
{
```

```
"preco": "7.4"
}
GET /coercion_test/_search
```

Observe que no campo `_source` os valores informados foram diferentes. Mas sabemos que, graças ao mecanismo de coercion, na etapa de indexação, ambos foram convertidos para o tipo float (tipo original do campo `preco`).

O comando a seguir falhará uma vez que não é possível fazer conversão de “7.4m” para float:

```
POST /coercion_test/_doc
{
  "preco": "7.4m"
}
```

Caso se deseje desabilitar o mecanismo de coerce, isto pode ser feito no momento de criação do índice da seguinte forma:

```
PUT /coercion_test
{
  "settings": {
    "index": {
      "mapping": {
        "coerce": "false"
      },
      "number_of_shards": "1",
      "number_of_replicas": "0"
    }
  },
  "mappings": {
    "properties": {
      "preco": {
        "type": "float"
      }
    }
  }
}
```

```
}  
}
```

O efeito prático de desabilitar o coerce será que a inserção de 7.4 (float) continuará funcionando, enquanto que a inserção de “7.4” (string) será barrada. Refaça os comandos POST vistos para testar o comportamento.

Uma dúvida interessante surge ao se tentar inserir documentos contendo mais campos em relação aos previstos no mapping original, como no exemplo a seguir:

```
POST /coercion_test/_doc  
{  
  "preco": 7.4,  
  "quantidade": 10  
}
```

Observe que o mecanismo de coercion apenas “barra” conversão de tipos. Ele não impede a criação de novos campos, de modo que o exemplo acima funcionará normalmente. Após a execução do mesmo, é possível consultar o mapping alterado:

```
GET /coercion_test
```

Retornará:

```
"mappings": {  
  "properties": {  
    "preco": {  
      "type": "float"  
    },  
    "quantidade": {  
      "type": "long"  
    }  
  }  
},
```

Como o campo quantidade foi inferido como sendo do tipo long, a seguinte tentativa falhará, uma vez que precisaria de conversão de string para long.

```
POST /coercion_test/_doc
```

```
{
  "preco": 7.4,
  "quantidade": "10"
}
```

E caso desejemos barrar ambas as ações: coercion e criação de novos campos no mapping?
O exemplo a seguir aplica ambas as características no índice.

```
PUT /coercion_test
```

```
{
  "settings": {
    "index": {
      "mapping": {
        "coerce": "false"
      },
      "number_of_shards": "1",
      "number_of_replicas": "0"
    }
  },
  "mappings": {
    "dynamic": "strict",
    "properties": {
      "preco": {
        "type": "float"
      }
    }
  }
}
```

A inserção de dynamic: strict, fará com que o seguinte exemplo falhe:

```
POST /coercion_test/_doc
```

```
{  
  "preco": 7.4,  
  "quantidade": 10  
}
```

Existem ainda mais possibilidades do dynamic, conforme pode ser visto na documentação oficial do Elastic:

`true`

New fields are added to the mapping (default).

`runtime`

New fields are added to the mapping as [runtime fields](#). These fields are not indexed, and are loaded from `_source` at query time.

`false`

New fields are ignored. These fields will not be indexed or searchable, but will still appear in the `_source` field of returned hits. These fields will not be added to the mapping, and new fields must be added explicitly.

`strict`

If new fields are detected, an exception is thrown and the document is rejected. New fields must be explicitly added to the mapping.

Fonte: <https://www.elastic.co/docs/reference/elasticsearch/mapping-reference/dynamic>

Aula – 23 de março de 2026

```
PUT /wikipedia
```

```
{  
  "settings": {  
    "number_of_shards": 4,  
    "number_of_replicas": 1,  
    "index.mapping.coerce": false,  
  }  
}
```

```
"analysis": {
  "analyzer": {
    "analyzer_for_content": {
      "type": "custom",
      "char_filter": [ ],
      "tokenizer": "standard",
      "filter": [
        "asciifolding",
        "lowercase",
        "snowball"
      ]
    }
  }
},
"mappings": {
  "dynamic": "strict",
  "properties": {
    "title": {
      "type": "text",
      "analyzer": "analyzer_for_content"
    },
    "url": {
      "type": "keyword",
      "doc_values": false,
      "index": false
    },
    "content": {
      "type": "text",
      "analyzer": "analyzer_for_content"
    }
  }
},
```

```

    "dt_creation":{
      "type": "date"
    },
    "reading_time":{
      "type": "integer"
    }
  }
}
}
}

```

Neste momento, faça o download do arquivo wiki.json (disponível no Google Classroom), e execute o seguinte comando no terminal para importá-lo.

```
curl -H "Content-Type: application/x-ndjson" -XPOST https://localhost:9200/wikipedia/_bulk --data-binary "@wiki.json" --user "elastic:user123" --insecure
```

Começaremos submetendo uma primeira query ao nosso novo índice:

```

GET /wikipedia/_search
{
  "query": {
    "match": {
      "content": "square root"
    }
  }
}

```

A consulta acima retornará documentos contendo quaisquer das palavras (square ou root). Isto porque o operador padrão ao se processar queries é o OU (OR). Existem ainda outras variações possíveis de uma query, que serão exibidas a seguir.

<pre> GET /wikipedia/_search { "query": { "match": { "content": "square root" } } } </pre>	<pre> GET /wikipedia/_search { "query": { "match": { "content": { "query": "square root", </pre>	<pre> GET /wikipedia/_search { "query": { "match_phrase": { "content": "square root" } } } </pre>
--	--	---

<pre> } } </pre>	<pre> "operator": "and" } } } </pre>	<pre> } } </pre>
------------------	--	------------------

A consulta à esquerda é a mesma já explicada anteriormente. No centro, temos a consulta modificada para utilizar o operador E (AND) em vez do OU (OR), que é o padrão. Note como isso reduz a quantidade de resultados retornados. Por fim, na coluna à direita, a busca é realizada usando `match_phrase`, o que significa que apenas documentos contendo exatamente a frase `square root` serão retornados. Essa abordagem é ainda mais restritiva que as anteriores, resultando em um número ainda menor de documentos encontrados.

A seguir temos um exemplo caso desejemos buscar em mais de um campo:

```
GET /wikipedia/_search
{
  "query": {
    "multi_match": {
      "query": "square root",
      "fields": ["content", "title"]
    }
  }
}
```

No exemplo acima, foram utilizados os campos *content* e *title* como alvo da query executada. Os exemplos a seguir retornam quantidade de resultados diferentes, ilustrando a situação onde o acréscimo do campo `title` na busca acrescentou mais um resultado retornado pelo Elasticsearch.

```
GET /wikipedia/_search
{
  "query": {
    "multi_match": {
      "query": "normalized google distance",
      "fields": ["content", "title"]
    }
  }
}
```

```
}
```

GET /wikipedia/_search

```
{
  "query": {
    "match": {
      "content": "normalized google distance"
    }
  }
}
```

Exercício - Analise a seguinte consulta, e faça alterações para compreender melhor seu funcionamento. Em seguida, faça o que se pede. A correção será feita na aula do dia 24/03.

GET /wikipedia/_search

```
{
  "query": {
    "bool": {
      "must": [
        {
          "match": {
            "content": {
              "query": "square root",
              "operator": "and"
            }
          }
        }, {
          "match_phrase": {
            "content": "square root of"
          }
        }
      ]
    }
  }
}
```

```

    "should": [
      {
        "match": {
          "title": "square root"
        }
      }
    ]
  }
}
}

```

Proponha uma consulta usando de boolean query de modo o resultado seja semelhante ao que obtemos do Google ao submeter:

the treap and the "randomized binary" search tree

Aula – 24 de março de 2026

```

GET /wikipedia/_search
{
  "query": {
    "bool": {
      "must": [
        {
          "match_phrase": {
            "content": "randomized binary"
          }
        }
      ],
      "should": [
        {
          "match": {
            "content": "treap search"
          }
        }
      ]
    }
  }
}

```

```

    }
  },
  {
    "match": {
      "content": "probability distribution"
    }
  }
]
}
}
}

```

Aula – 30 de março de 2026

Conforme visto na última aula, a busca booleana possui três possibilidades internas:

- must: obrigatório! - termos colocados na cláusula must deverão ser encontrados (lembrando que, internamente em cada busca, o padrão entre os termos é OR).
- should: se tiver, melhor! - termos colocados na cláusula should não são obrigatórias, mas caso aconteçam, aumentam o score do resultado cujo should foi bem sucedido.
- must not: obrigatório “negativo”! - termos colocadas em must not não podem estar presentes nos resultados.

Modificando o valor de ranking (score) dos documentos

Começemos hoje com a seguinte consulta:

```

GET /wikipedia/_search
{
  "query": {
    "bool": {
      "must": [
        {
          "match": {
            "content": "randomized binary"
          }
        }
      ]
    }
  }
}

```

```

    }
  ],
  "should": [
    {
      "match": {
        "content": {
          "query": "treap search"
        }
      }
    }
  ]
}
}
}

```

Suponha uma situação onde tenhamos várias cláusulas dentro do *should* (ou de algum outro bloco como o *must* ou *must not*). Neste cenário, não necessariamente todas as cláusulas internas do comando deverão ter o mesmo peso (importância) ao contribuir para o valor de score do resultado cujo *match* ocorreu. A maneira de modificar o peso (importância) de uma cláusula é através do comando *boost*, ilustrado a seguir.

GET /wikipedia/_search

```

{
  "query": {
    "bool": {
      "must": [
        {
          "match": {
            "content": "randomized binary"
          }
        }
      ]
    },
    "should": [

```

```

{
  "match": {
    "content": {
      "query": "treap search",
      "boost": 1.0
    }
  }
}
]
}
}
}

```

Um valor de **boost** igual a 1 manterá o **score** original. **Valores menores que 1** reduzirão o **score**, enquanto **valores maiores que 1** aumentarão o **score** calculado inicialmente.

Por fim, uma última consideração sobre os blocos **must** e **should**. Caso uma consulta tenha somente o bloco **should** (sem o **must**), o **should** se tornará obrigatório. Assim, as queries a seguir são equivalentes.

<pre> GET /wikipedia/_search { "query": { "bool": { "must": [{ "match": { "content": "randomized binary" } }], "should": [{ "match_phrase": { "content": "randomized binary" } }] } } } </pre>	<pre> GET /wikipedia/_search { "query": { "bool": { "should": [{ "match": { "content": "randomized binary" } }, { "match_phrase": { "content": "randomized binary" } }] } } } </pre>
--	--

É possível ainda definir uma quantidade mínima de **shoulds** que devem ser obrigatórios em uma query. A maneira de se fazer isso é através da propriedade **minimum_should_match**. A seguir temos um exemplo da mesma, que torna todos os elementos do **should** obrigatórios.

```
GET /wikipedia/_search
```

```
{
  "query": {
    "bool": {
      "should": [
        {
          "match": {
            "content": "randomized binary"
          }
        },
        {
          "match_phrase": {
            "content": "randomized binary"
          }
        }
      ],
      "minimum_should_match": 2
    }
  }
}
```

As consultas a seguir farão uso da cláusula **range**, de modo a filtrarmos (ou priorizarmos) os resultados contendo uma determinada faixa para o campo *reading_time*. Observe que o *range* entra como mais um elemento dentro de um dos blocos (**must** ou **should**), de modo a definir se o limite deve ser mandatório ou opcional.

GET /wikipedia/_search { "query": {	GET /wikipedia/_search { "query": {
---	---

<pre> "bool": { "must": [{ "match": { "content": "randomized binary" } }, { "range": { "reading_time": { "lt": 8 } } }], "should": [{ "match_phrase": { "content": "randomized binary" } }] } </pre>	<pre> "bool": { "must": [{ "match": { "content": "randomized binary" } }], "should": [{ "match_phrase": { "content": "randomized binary" } }, { "range": { "reading_time": { "lt": 8 } } }] } </pre>
--	--

Ao se utilizar o **range** no bloco **should** (opcional), alguns documentos retornados poderão não casar com esta condição, mas os documentos que casarem com a condição do **range** (neste caso, *reading_time* < 8) receberão um valor de rank += 1.0.

Outra forma de filtrar resultados por *reading_time* é utilizando a propriedade **filter**. Nesse caso, seu funcionamento é semelhante ao uso de **range** dentro do bloco **must**, pois o conteúdo de **filter** também é obrigatório. A diferença, no entanto, é que a propriedade **filter** não altera o valor original do **score**.

GET /wikipedia/_search

```

{
  "query": {
    "bool": {
      "must": [
        {

```

```

        "match": {
            "content": "binary search"
        }
    },
    ],
    "filter": [
        {
            "range": {
                "reading_time": {
                    "lte": 8
                }
            }
        }
    ]
}
}
}

```

Named queries

O Elasticsearch permite ainda acrescentar o parâmetro **_name**, com um nome (que pode ser definido pelo usuário). Neste caso, os resultados incluirão um campo extra, chamado **"matched_queries"**, contendo a informação de quais queries casaram com aquele resultado. A seguir temos a query modificada, seguida pelo print da tela de exemplo.

GET /wikipedia/_search

```

{
  "query": {
    "bool": {
      "must": [
        {
          "match": {
            "content": {

```

```

        "query": "randomized binary",
        "_name": "must"
    }
}
},
], "should": [
{
    "match_phrase": {
        "content": {
            "query": "randomized binary",
            "_name": "should"
        }
    }
}
]
}
}
}

```

Exemplo de resultado contendo ambos os “named queries” acima.

```

"content": ""In the mathematical theory of probability, Janson's
inequality is a collection of related inequalities giving an
exponential bound on the probability of many related events happening
simultaneously by their pairwise dependence. Informally Janson's
inequality involves taking a sample of many independent random binary
variables, and a set of subsets of those variables and bounding the
probability that the sample will contain any of those subsets by their
pairwise correlation. Statement Let <math>\{X_i\}_{i=1}^n</math> be a set of
independent random binary variables, and let <math>A_1, A_2, \dots, A_m</math>
be a collection of subsets of <math>\{1, 2, \dots, n\}</math>. Let
<math>p_i = P(X_i = 1)</math> and <math>p_A = \prod_{i \in A} p_i</math>. Then,
<math>P(\sum_{i \in A} X_i \geq t) \leq \exp(-t^2 / (2 \sum_{i \in A} p_i))</math>."
",
"dt_creation": "2015-07-08",
"reading_time": 3
},
"matched_queries": [
  "should",
  "must"
]

```

Por fim, faremos o filtro utilizando de **range** (embora o uso com **filter** neste caso é semelhante ao que já vimos no anteriormente). O exemplo de **range** a seguir filtra por data (**dt_creation**), e não por **reading_time**.

GET /wikipedia/_search

```

{
  "query": {
    "bool": {
      "must": [
        {
          "match": {
            "content": "binary search"
          }
        }, {
          "range": {
            "dt_creation": {
              "gte": "2018-01-01"
            }
          }
        }, {
          "range": {

```

Aula – 31 de março de 2026

Hoje veremos como limitar os dados retornados internamente no campo **_source**. Suponha que desejemos retornar apenas os campos **title** e **url** de uma busca. A query a seguir faz exatamente isto.

```
GET /wikipedia/_search
```

```
{
  "_source": ["title", "url"],
  "query": {
    "match": {
      "content": "randomized binary"
    }
  }
}
```

O Elasticsearch fornece suporte ainda a outros formatos no retorno (ao invés do JSON, que é o padrão). A seguir vemos como formatar os resultados em formato YAML:

```
GET /wikipedia/_search?format=yaml
```

```
{
  "_source": ["title", "url"],
  "query": {
    "match": {
```

```

        "content": "randomized binary"
    }
}
}

```

Outra possibilidade ainda é definirmos a quantidade de resultados retornados em uma consulta. A maneira de fazer isto é através do parâmetro **size**, visto a seguir:

```

GET /wikipedia/_search
{
  "_source": ["title", "url"],
  "size": 20,
  "query": {
    "match": {
      "content": "randomized binary"
    }
  }
}

```

Calculando o total de páginas

Para fazermos a paginação de resultados utilizaremos do cálculo de duas variáveis simples:

Calcular o total de páginas:

$$total_paginas = teto(total_resultados / tamanho_pagina)$$

Exemplo:

Se uma busca retornou 137 resultados, considerando que cada página tenha 10 resultados, teremos então 14 páginas de resultados.

teto é uma função que retorna o menor inteiro maior ou igual a x. Ou seja, o teto(13,7) = 14.

Obter somente os resultados de determinada página:

$$from = ((numero_pagina - 1) * tamanho_pagina)$$

Para se obter os resultados da página 1, considerando-se 10 resultados por página, e seguindo a fórmula acima, ficará:

$$from = (10 * (1-1)) \rightarrow 0$$

Para a página 2:

$$from = (10 * (2-1)) \rightarrow 10$$

Exemplo → a query a seguir retornará os resultados relativos à página 3 (from = 20):

```
GET /wikipedia/_search
{
  "_source": ["title", "url"],
  "from": 20,
  "size": 10,
  "query": {
    "match": {
      "content": "randomized binary"
    }
  }
}
```

Ordenação de resultados

Por padrão o Elasticsearch utiliza uma medida para calcular a ordenação dos resultados, chamada de Okapi BM25. Contudo, caso o usuário deseje alterar isso tomando como base algum campo específico, é possível. Para isso, basta utilizar o parâmetro sort, inserido depois da query.

```
GET /wikipedia/_search
{
  "query": {
    "bool": {
      "must": [
        {
          "match": {
            "content": {
              "query": "randomized binary"
            }
          }
        }
      ]
    }
  }
}
```

```

    }
  }
]
}
},
"sort": [
  {
    "reading_time": {
      "order": "desc"
    }
  }
]
}

```

O parâmetro `sort` é um array (vetor), como demonstrado acima. Isso permite definir múltiplos critérios de ordenação para uma única consulta. O Elasticsearch respeita a ordem definida no vetor: caso haja empate no primeiro critério, o motor de busca utilizará o segundo, e assim sucessivamente. Na consulta a seguir, os resultados são ordenados primeiramente pelo `reading_time` em ordem ascendente; em caso de valores idênticos, o desempate será definido pelo `_score` (baseado no algoritmo Okapi BM25).

`GET /wikipedia/_search`

```

{
  "query": {
    "bool": {
      "must": [
        {
          "match": {
            "content": {
              "query": "randomized binary"
            }
          }
        }
      ]
    }
  }
}

```

```

    }
  ]
}
},
"sort": [
  {
    "reading_time": {
      "order": "asc"
    }
  },
  {
    "_score": {
      "order": "desc"
    }
  }
]
}

```

Acrescentando graus de liberdade ao match_phrase

A cláusula **match_phrase** traz a possibilidade de se acrescentar o parâmetro **slop**, que insere na busca um grau de liberdade com relação tanto à ordem dos termos, quanto à ocorrência de outros termos entre os mesmos. Vejamos o exemplo a seguir...

GET /wikipedia/_search

```

{
  "_source": ["title", "content"],
  "query": {
    "match_phrase": {
      "content": {
        "query": "Kolmogorov equations",
        "slop": 1
      }
    }
  }
}

```

```
}  
}  
}
```

O exemplo acima retornará resultados contendo a frase *Kolmogorov equations*, podendo ter uma palavra entre ambas. Por exemplo, *Kolmogorov backward equations*.

A seguir temos um exemplo um pouco mais complexo de uso de busca booleana com o parâmetro slop.

GET /wikipedia/_search

```
{  
  "_source": ["title", "content"],  
  "query": {  
    "bool": {  
      "must": [  
        {  
          "match": {  
            "content": "Kolmogorov equations"  
          }  
        }  
      ],  
      "should": [  
        {  
          "match_phrase": {  
            "content": {  
              "query": "Kolmogorov equations",  
              "slop": 1  
            }  
          }  
        }  
      ]  
    }  
  }  
}
```

```
}  
}
```

Aula – 6 de abril de 2026

A consulta a seguir trará as palavras Kolmogorov equations devidamente destacadas no retorno do Elasticsearch.

Dica: Caso deseje visualizar apenas o resumo, modifique o campo `_source` com o valor *empty*. Esta ação fará com que o source retorne vazio, e apenas o bloco *highlight* possua os dados internamente.

```
GET /wikipedia/_search
```

```
{  
  "_source": ["title", "url"],  
  "query": {  
    "bool": {  
      "must": [  
        {  
          "match": {  
            "content": "Kolmogorov equations"  
          }  
        }  
      ],  
      "should": [  
        {  
          "match_phrase": {  
            "content": "Kolmogorov equations"  
          }  
        }  
      ]  
    }  
  },  
}
```

```
"highlight": {  
  "fields": {  
    "content": {}  
  }  
}
```

Perceba que, por padrão, o Elasticsearch retornará os termos a serem destacados contidos entre tags `<em>` `</em>`. São retornados ainda alguns fragmentos por documento (padrão até 5 fragmentos). É importante destacar que todas estas características são parametrizadas. A seguir temos um exemplo, repetindo-se a consulta, mas com algumas modificações feitas (modificação das tags de abertura e fechamento, definição da quantidade de segmentos retornados e tamanho do segmento).

```
GET /wikipedia/_search
```

```
{  
  "_source": [  
    "title",  
    "url"  
  ],  
  "query": {  
    "bool": {  
      "must": [  
        {  
          "match": {  
            "content": "Kolmogorov equations"  
          }  
        }  
      ],  
      "should": [  
        {  
          "match_phrase": {  
            "content": "Kolmogorov equations"  
          }  
        }  
      ]  
    }  
  }  
}
```

```

    }
  }
]
}
},
"highlight": {
  "pre_tags": ["<strong>"],
  "post_tags": ["</strong>"],
  "number_of_fragments": 1,
  "fragment_size": 400,
  "fields": {
    "content": {}
  }
}
}

```

Enquanto o parâmetro query define quais documentos compõem o resultado, o bloco highlight permite customizar o critério de marcação visual por meio de uma sub-query. Abaixo, demonstramos um caso onde os critérios divergem: a busca utiliza o operador lógico OR para retornar documentos com 'Kolmogorov' ou 'equations', mas o realce é configurado como match_phrase. O resultado prático é que apenas a ocorrência exata da expressão será destacada, ignorando palavras soltas.

Por fim, é possível aplicar diversas variações, como por exemplo, o acréscimo de slop no bloco de highlight:

```

GET /wikipedia/_search
{
  "_source": ["title", "url"],
  "query": {
    "bool": {
      "must": [
        {
          "match": {

```

```

        "content": "Kolmogorov equations"
    }
}
],
"should": [
    {
        "match_phrase": {
            "content": "Kolmogorov equations"
        }
    }
]
}
},
"highlight": {
    "pre_tags": ["<strong>"],
    "post_tags": ["</strong>"],
    "number_of_fragments": 1,
    "fragment_size": 400,
    "fields": {
        "content": {
            "highlight_query": {
                "bool": {
                    "must": [
                        {
                            "match_phrase": {
                                "content": {
                                    "query": "Kolmogorov equations",
                                    "slop": 1
                                }
                            }
                        }
                    ]
                }
            }
        }
    }
}

```

```
]
}
}
}
}
}
}
```

A seguir temos exemplos para os três cenários:

- highlight igual ao query (Kolmogorov <OR> equations):

Kolmogorov equations (continuous-time Markov chains) In mathematics and statistics, in the context of Markov processes, the **Kolmogorov equations**, including **Kolmogorov forward equations** and **Kolmogorov backward equations**, are a pair of systems of differential **equations** that describe the time evolution of the process's distribution.

- Destaque contendo apenas o bloco should (phrase **Kolmogorov equations**):

Kolmogorov equations (continuous-time Markov chains) In mathematics and statistics, in the context of Markov processes, the **Kolmogorov equations**, including Kolmogorov forward equations and Kolmogorov backward equations, are a pair of systems of differential equations that describe the time evolution of the process's distribution.

- Destaque contendo apenas o bloco should (phrase **Kolmogorov equations** com slop 1):

Kolmogorov equations (continuous-time Markov chains) In mathematics and statistics, in the context of Markov processes, the **Kolmogorov equations**, including **Kolmogorov forward equations** and **Kolmogorov backward equations**, are a pair of systems of differential equations that describe the time evolution of the process's distribution.

Fuzzy match query

A fuzzy match query faz uso do algoritmo de distância de Levenshtein para poder encontrar documentos, ainda que o usuário cometa um pequeno erro ao digitar a query. A distância de Levenshtein computa quantas operações (edição, inserção, ou deleção) precisam ser feitas de modo que uma String seja convertida em outra.

Assim, ao se setar o parâmetro fuzziness como auto, o valor será definido automaticamente pelo Elasticsearch tomando como referência o tamanho da query. Os valores aceitos para fuzziness são: 0, 1 ou 2.

Acrescentamos ainda um bloco highlight, de modo a ser possível ver exatamente qual o termo retornado como match pelo Elasticsearch.

```
GET /wikipedia/_search
```

```
{
  "_source": "empty",
  "query": {
    "bool": {
      "must": [
        {
          "match": {
            "content": {
              "query": "equation",
```

```

        "fuzziness": 1
      }
    }
  }
]
}
},
"highlight": {
  "number_of_fragments": 1,
  "fragment_size": 400,
  "fields": {
    "content": {}
  }
}
}

```

Aula – 07 de abril de 2026

GET /wikipedia/_search

```

{
  "size": 0,
  "aggs": {
    "total_reading_time": {
      "sum": {
        "field": "reading_time"
      }
    },
    "avg_reading_time": {
      "avg": {
        "field": "reading_time"
      }
    },
  },

```

```
"max_reading_time":{
  "max": {
    "field": "reading_time"
  }
},
"min_reading_time":{
  "min": {
    "field": "reading_time"
  }
},
"count_reading_time":{
  "value_count": {
    "field": "reading_time"
  }
}
}
}
```

GET /wikipedia/_search

```
{
  "size": 0,
  "aggs": {
    "stats_reading_time": {
      "stats": {
        "field": "reading_time"
      }
    }
  }
}
```

GET /wikipedia/_search

```
{
  "size": 0,
  "aggs": {
    "cardinality_reading_time": {
      "cardinality": {
        "field": "reading_time"
      }
    }
  }
}
```

Aula – 13 de abril de 2026

O comando `reindex` é um comando bastante útil quando desejamos criar um novo index tomando como base o conteúdo de um índice já existente.

Começaremos criando um novo índice, com a mesma estrutura do índice `wikipedia` (já existente). Alteramos apenas a quantidade de shards (para 1), e a quantidade de réplicas (para 0).

PUT `/wikipedia_v2`

```
{
  "settings": {
    "number_of_shards": 1,
    "number_of_replicas": 0,
    "index.mapping.coerce": false,
    "analysis": {
      "analyzer": {
        "analyzer_for_content": {
          "type": "custom",
          "char_filter": [],
          "tokenizer": "standard",
          "filter": [
```

```
        "asciifolding",
        "lowercase",
        "snowball"
    ]
}
}
},
},
"mappings": {
    "dynamic": "strict",
    "properties": {
        "title": {
            "type": "text",
            "analyzer": "analyzer_for_content"
        },
        "url": {
            "type": "keyword",
            "doc_values": false,
            "index": false
        },
        "content": {
            "type": "text",
            "analyzer": "analyzer_for_content"
        },
        "dt_creation": {
            "type": "date"
        },
        "reading_time": {
            "type": "integer"
        },
        "label": {
```

```

        "type": "keyword"
    }
}
}
}

```

O script a seguir cria a lógica para que o novo campo label seja devidamente preenchido. O preenchimento desejado será com base no campo reading_time, de acordo com a seguinte lógica:

- reading_time <= 5 → rápido
- 5 < reading_time <= 10 → médio
- 10 < reading_time <= 15 → demorado

```

PUT _ingest/pipeline/pipeline_leitura_wikipedia
{
  "description": "Classifica documentos pelo tempo de leitura",
  "processors": [
    {
      "script": {
        "lang": "painless",
        "source": """
          if (ctx.reading_time != null) {
            int t = ctx.reading_time;
            if (t <= 5) {
              ctx.label = 'rápido';
            } else if (t <= 10) {
              ctx.label = 'médio';
            } else {
              ctx.label = 'demorado';
            }
          }
        """
      }
    }
  ]
}

```

```

        ""
    }
}
]
}

```

Comando reindex:

```

POST _reindex?wait_for_completion=false
{
  "source": {
    "index": "wikipedia"
  },
  "dest": {
    "index": "wikipedia_v2",
    "pipeline": "pipeline_leitura_wikipedia"
  }
}

```

O parâmetro `wait_for_completion=false` fará com que o Kibana não fique bloqueado aguardando pelo seu término. O retorno deste comando será um valor de task, que podemos utilizar depois para acompanhar o progresso do reindex através do GET `/_task`. A seguir temos um exemplo.

```

GET /_tasks/<id_task>

```

Exemplo:

```

GET /_tasks/qNSQ-08KQLm8uFEca02d0w:42029

```

Mecanismo de sugestão

A seguir trabalharemos com o mecanismo de Suggest (sugestão) presente no Elasticsearch. Em mecanismos de busca, é comum que usuários cometam erros de digitação ou ortografia ao inserir termos de pesquisa. Para lidar com esses casos e melhorar a experiência de busca, o Elasticsearch oferece um recurso chamado Suggest.

A consulta a seguir exemplifica o uso do mecanismo de sugestão (suggest) do Elasticsearch. Este mecanismo analisa as palavras inseridas na consulta e, caso identifique termos com baixo índice de correspondência (match) nos documentos indexados, gera possíveis sugestões de correção para essas palavras. O objetivo é auxiliar o usuário a encontrar os resultados desejados, mesmo que tenha digitado o termo incorretamente.

De maneira geral, o fluxo de funcionamento é o seguinte:

A palavra enviada na consulta é comparada com o vocabulário presente no índice.

Se o Elasticsearch detectar que a palavra não possui ocorrências significativas (ou nenhuma), ele aplicará algoritmos de similaridade para sugerir termos alternativos.

As sugestões são retornadas ao usuário, que pode optar por reformular a busca utilizando uma das palavras sugeridas.

Este recurso é fundamental para a construção de sistemas de busca mais robustos e tolerantes a erros humanos, melhorando significativamente a precisão e a relevância dos resultados apresentados.

POST /wikipedia/_search

```
{
  "suggest": {
    "YOUR_SUGGESTION": {
      "text": "The Schrödiger equation is a pargial differential equation fo the wqve funxtion of
quantum mechanics.",
      "term": {
        "field": "content",
        "size": 1
      }
    }
  }
}
```

Observe que, para fins de teste do mecanismo de sugestão, algumas palavras foram propositalmente digitadas de forma incorreta.

O Elasticsearch, ao processar a consulta com o recurso de suggest, identificará essas palavras com baixo índice de correspondência no índice e retornará, para cada uma delas, um vetor de opções (options).

Cada vetor options conterá possíveis sugestões de correção, ordenadas de acordo com a relevância ou a similaridade em relação à palavra original.

```
"text": "schrodig",
"offset": 4,
"length": 10,
"options": [
  {
    "text": "schroding",
    "score": 0.875,
    "freq": 96
  }
]
```

Ao analisarmos a figura acima vemos que tanto o valor de text quanto o valor de options.text se baseiam nas informações vindas do índice invertido. As correções são válidas, mas não podemos retornar ao usuário na interface que uma sugestão para Schrödiger (conforme foi digitado) seja schroding, que é o resultado de Schrödinger após ter passado pelos filtros de:

```
"filter": [
  "asciifolding",
  "lowercase",
  "snowball"
]
```

Aula – 27 de abril de 2026

Proposta do trabalho final.

Endereço do repositório: https://github.com/fbgonzaga/elasticsearch_example

Consulta via Postman:

The screenshot shows the Postman interface for an HTTP GET request to `http://localhost:8080/v1/search?query=math&page=3`. The request is successful, returning a `200 OK` status with a response time of 47 ms and a body size of 5.62 KB. The response body is displayed in JSON format, showing a list of search results for the query "math".

Key	Value	Description
query	math	
page	3	

```
[
  {
    "title": "Polykay",
    "url": "https://en.wikipedia.org/wiki/Polykay",
    "abs": "Polykay In statistics a polykay or generalised kstatistic denoted krs is a statistic defined as a linear combination of sample moments Etymology The word polykay was coined by American mathematician John Tukey in from poly many or much and kay the phonetic spelling of the letter k as in kstatistic References Tukey J W Keeping MomentLike Computations Simple Ann Math Stat "
```

Site para visualização do api.yml:

<https://editor.swagger.io/>